```
import streamlit as st
import pandas as pd
import numpy as np
import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots
from datetime import datetime, timedelta, date
from streamlit_gsheets import GSheetsConnection

# ==========================================
# 1. 系統設定與網頁配置
# ==========================================
st.set_page_config(page_title="個人智慧投資紀錄簿", layout="wide",
initial_sidebar_state="expanded")

# 建立 Google Sheets 連結物件
conn = st.connection("gsheets", type=GSheetsConnection)

# 安全檢查：確保 Secrets 設定正確
if "connections" not in st.secrets or "gsheets" not in st.secrets["connections"] or
"spreadsheet" not in st.secrets["connections"]["gsheets"]:
    st.error("⚠️ 偵測到雲端設定錯誤！請檢查 Streamlit Cloud 控制台中的 Secrets 設定。")
    st.stop()

# 初始化 API 遭阻擋的狀態標記
if "is_api_blocked" not in st.session_state:
    st.session_state["is_api_blocked"] = False

# ==========================================
# 🏦 自動化：動態貸款餘額扣除計算函式
# ==========================================
def calculate_remaining_loans(current_date):
    """
    依據當前日期，採用獨立歷史起算點，自動計算兩筆貸款的累計已還款期數與剩餘金額
    """
    l1_base = 1039721
    l1_day = 20
    l1_pay = 12797
    l1_base_date = date(2024, 4, 20)

    l2_base = 2016662
    l2_day = 10
    l2_pay = 18872
    l2_base_date = date(2026, 4, 10)

    l1_payments = 0
    start_yr_1, start_mo_1 = l1_base_date.year, l1_base_date.month
    end_yr, end_mo = current_date.year, current_date.month
```

```
    current_yr, current_mo = start_yr_1, start_mo_1
    while (current_yr < end_yr) or (current_yr == end_yr and current_mo <= end_mo):
        pay_date_l1 = date(current_yr, current_mo, l1_day)
        if l1_base_date <= pay_date_l1 <= current_date:
            l1_payments += 1
        if current_mo == 12:
            current_mo = 1
            current_yr += 1
        else:
            current_mo += 1

    l2_payments = 0
    start_yr_2, start_mo_2 = l2_base_date.year, l2_base_date.month

    current_yr, current_mo = start_yr_2, start_mo_2
    while (current_yr < end_yr) or (current_yr == end_yr and current_mo <= end_mo):
        pay_date_l2 = date(current_yr, current_mo, l2_day)
        if l2_base_date <= pay_date_l2 <= current_date:
            l2_payments += 1
        if current_mo == 12:
            current_mo = 1
            current_yr += 1
        else:
            current_mo += 1

    l1_rem = max(0, l1_base - (l1_payments * l1_pay))
    l2_rem = max(0, l2_base - (l2_payments * l2_pay))

    return l1_rem, l2_rem, l1_payments, l2_payments

# ==========================================
# 🔒 系統安全登入驗證機制 (Session State)
# ==========================================
if "logged_in" not in st.session_state:
    st.session_state["logged_in"] = False

if not st.session_state["logged_in"]:
    st.markdown("<h2 style='text-align: center;'>🔒 智慧投資紀錄簿 - 系統登入</h2>",
unsafe_allow_html=True)
    st.write("")
    col_space1, col_login, col_space2 = st.columns([1, 2, 1])
    with col_login:
        with st.form("login_form", clear_on_submit=False):
            username_input = st.text_input("請輸入管理員帳號：")
            password_input = st.text_input("請輸入密碼：", type="password")
            submit_login = st.form_submit_button("🚀 安全登入", use_container_width=True)
```

```
            if submit_login:
                try:
                    df_creds = conn.read(worksheet="user_credentials", ttl=0)
                    df_creds = df_creds.dropna(subset=["帳號", "密碼"])
                    match = df_creds[
                        (df_creds["帳號"].astype(str) == username_input) &
                        (df_creds["密碼"].astype(str) == password_input)
                    ]
                    if not match.empty:
                        st.session_state["logged_in"] = True
                        st.session_state["username"] = username_input
                        st.success("🎉 登入成功！正在跳轉...")
                        st.rerun()
                    else:
                        st.error("❌ 帳號或密碼錯誤，請重新確認！")
                except Exception as e:
                    st.error(f"❌ 驗證失敗。錯誤: {e}")
        st.stop()

# ==========================================
# 2. 線上即時股價抓取
# ==========================================
def fetch_realtime_prices(tickers):
    prices = {}
    valid_tickers = list(set([str(t).strip() for t in tickers if t and str(t).strip() != '現金']))

    fallback_prices = {
        "00631L.TW": 32.17,
        "00685L.TW": 10.54,
        "00662.TW": 118.85,
        "QQQM": 290.68,
        "00865B.TW": 49.25
    }

    if not valid_tickers:
        return prices

    try:
        import yfinance as yf
        data = yf.download(valid_tickers, period='5d', group_by='ticker', progress=False)
        if not data.empty:
            has_zero = False
            for ticker in valid_tickers:
                try:
                    df_ticker = data if len(valid_tickers) == 1 else data[ticker]
                    if 'Close' in df_ticker.columns:
                        closes = df_ticker['Close'].dropna()
                        if not closes.empty and float(closes.iloc[-1]) > 0:
```

```
                    prices[ticker] = round(float(closes.iloc[-1]), 2)
                else:
                    prices[ticker] = fallback_prices.get(ticker, 0.0)
                    has_zero = True
            else:
                prices[ticker] = fallback_prices.get(ticker, 0.0)
                has_zero = True
        except:
            prices[ticker] = fallback_prices.get(ticker, 0.0)
            has_zero = True
        st.session_state["is_api_blocked"] = has_zero
    else:
        st.session_state["is_api_blocked"] = True
        for ticker in valid_tickers:
            prices[ticker] = fallback_prices.get(ticker, 0.0)
except Exception:
    st.session_state["is_api_blocked"] = True
    for ticker in valid_tickers:
        prices[ticker] = fallback_prices.get(ticker, 0.0)

for ticker in valid_tickers:
    if prices.get(ticker, 0.0) <= 0:
        prices[ticker] = fallback_prices.get(ticker, 10.0)

return prices

@st.cache_data(ttl=60)
def get_usd_twd_rate():
    try:
        import yfinance as yf
        data = yf.download("USDTWD=X", period='5d', progress=False)
        if not data.empty and 'Close' in data.columns:
            closes = data['Close'].dropna()
            if not closes.empty and float(closes.iloc[-1]) > 0:
                return round(float(closes.iloc[-1]), 4)
        return 32.5
    except Exception:
        return 32.5

# ==========================================
# 3. 側邊導覽列與功能選單
# ==========================================
st.sidebar.title("🧭 投資導覽控制台")
st.sidebar.write(f"👤 目前使用者：`{st.session_state['username']}`")
menu = st.sidebar.radio("請選擇操作功能：", ["📊 投資總覽儀表板", "✍️ 每日資產動態輸入",
"⚙️ 投資標的持股管理"])

st.sidebar.markdown("---")
```

```
if st.sidebar.button("🔓 安全登出系統", use_container_width=True):
    st.session_state["logged_in"] = False
    st.session_state["username"] = None
    st.rerun()

# ========================================
# 功能一：📊 投資總覽儀表板
# ========================================
if menu == "📊 投資總覽儀表板":
    st.title("📊 個人即時投資動態儀表板 (Google Sheets 同步)")

    try:
        df_history = conn.read(worksheet="daily_asset_history", ttl=0)
        df_portfolio = conn.read(worksheet="portfolio_config", ttl=0)
    except Exception as e:
        st.error(f"❌ 無法讀取 Google Sheets！ 錯誤訊息: {e}")
        st.stop()

    df_history = df_history.dropna(subset=["日期"])
    df_portfolio = df_portfolio.dropna(subset=["標的名稱"])

    with st.spinner('正在獲取最新即時報價與匯率...'):
        current_prices = fetch_realtime_prices(df_portfolio['Yahoo代號'].tolist())
        usd_twd_rate = get_usd_twd_rate()

    st.sidebar.metric("💵 當前美金匯率 (USD/TWD)", f"${usd_twd_rate:.4f}")

    today_date = datetime.now().date()
    l1_remain, l2_remain, l1_pay_count, l2_pay_count =
calculate_remaining_loans(today_date)
    total_loan_balance = l1_remain + l2_remain

    # 🎯 核心修正點：判斷現價載入來源。如果 API 遭到風控，直接切換讀取 Google 表單的新
欄位
    # ⚠️ 請確保以下引號內的 '個股現價' 字眼與你 Google 試算表上的新欄位名稱完全一致！
    target_price_column = '個股現價'

    if st.session_state["is_api_blocked"] and target_price_column in df_portfolio.columns:
        df_portfolio['單位現價'] = pd.to_numeric(df_portfolio[target_price_column],
errors='coerce').fillna(0.0)
    else:
        df_portfolio['單位現價'] = df_portfolio['Yahoo代號'].map(current_prices).fillna(0.0)

    df_portfolio.loc[df_portfolio['Yahoo代號'] == '現金', '單位現價'] = 1.0

    def calculate_twd_market_value(row):
        ticker = str(row['Yahoo代號']).strip()
        price = float(row['單位現價'])
```

```
        qty = float(row['持有數量'])
        if ticker.endswith('.TW') or ticker == '現金':
            return price * qty
        else:
            return price * qty * usd_twd_rate

    df_portfolio['當前市值'] = df_portfolio.apply(calculate_twd_market_value, axis=1)

    total_market_value = df_portfolio['當前市值'].sum()
    total_cost = df_portfolio['投資成本'].sum()

    if not df_history.empty:
        df_history_sorted = df_history.copy()
        df_history_sorted['日期'] = pd.to_datetime(df_history_sorted['日期'])
        df_history_sorted = df_history_sorted.sort_values(by="日期")

        # 強制指定資料來源為 Google Sheets 最新一列做保底
        total_market_value = float(df_history_sorted['總資產金額'].iloc[-1])
        total_roi = float(df_history_sorted['每日報酬率'].iloc[-1])
        total_profit = total_market_value - total_cost
    else:
        total_profit = total_market_value - total_cost
        total_roi = (total_profit / total_cost) if total_cost > 0 else 0

    st.markdown("### ⚡ 快速同步控制區")
    col_btn, col_info = st.columns([1, 3])
    with col_btn:
        sync_clicked = st.button("🔄 立即同步最新資產至 Google 雲端",
use_container_width=True)
    with col_info:
        st.info("💡 雖然系統會定時自動抓取，但你可以隨時點擊按鈕，即時更新今日最新的資
產紀錄！")

    if sync_clicked:
        today_str = datetime.now().strftime("%Y-%m-%d")
        df_history['日期'] = df_history['日期'].astype(str)

        df_history_temp = df_history.copy()
        df_history_temp['日期'] = pd.to_datetime(df_history_temp['日期'])
        df_history_temp = df_history_temp.sort_values(by="日期")
        df_history_temp['日期'] = df_history_temp['日期'].dt.strftime("%Y-%m-%d")

        df_yesterday = df_history_temp[df_history_temp['日期'] < today_str]
        if not df_yesterday.empty:
            last_amount = float(df_yesterday['總資產金額'].iloc[-1])
            daily_diff = total_market_value - last_amount
        else:
            daily_diff = 0.0
```

```
        daily_roi = round(total_roi, 4)
        total_market_value_rounded = int(round(total_market_value))
        daily_diff_rounded = int(round(daily_diff))

        new_data = {
            "日期": today_str,
            "總資產金額": total_market_value_rounded,
            "每日增額": daily_diff_rounded,
            "每日報酬率": float(daily_roi)
        }

        if today_str in df_history['日期'].values:
            df_history.loc[df_history['日期'] == today_str, ["總資產金額", "每日增額", "每日報酬率
"]] = [
                total_market_value_rounded,
                daily_diff_rounded,
                float(daily_roi)
            ]
        else:
            df_history = pd.concat([df_history, pd.DataFrame([new_data])], ignore_index=True)

        with st.spinner('正在寫入 Google Sheets...'):
            conn.update(worksheet="daily_asset_history", data=df_history)
        st.success(f"🎉 成功同步！今日 ({today_str}) 資產已更新為整數型態。請重新整理網
頁！")

    st.markdown("---")

    # KPI 指標卡片
    col1, col2, col3, col4 = st.columns(4)
    col1.metric("當前總市值 (TWD)", f"${total_market_value:,.2f}")
    col2.metric("投資總成本", f"${total_cost:,.2f}")
    col3.metric("累積投資獲利", f"${total_profit:,.2f}", delta=f"{total_roi*100:.2f}% 報酬率")

    maintenance_rate = (total_market_value / total_loan_balance) if total_loan_balance > 0
else 0
    col4.metric("質押維持率", f"{maintenance_rate*100:.2f}%", delta="✅ 水位強韌" if
maintenance_rate > 1.6 else "⚠️ 需注意風險")

    st.markdown("### 🏦 剩餘貸款與還款明細")
    loan_col1, loan_col2, loan_col3 = st.columns(3)
    with loan_col1:
        st.info(f"**第一筆貸款 (每月 20 號還款)**\n* 剩餘金額：`${l1_remain:,.0f}` 元\n* 月還款
額：`${12797:,.0f}` 元\n* 累計已還款：`{l1_pay_count}` 期\n*(基準起算點：2024-04-20)*")
    with loan_col2:
        st.info(f"**第二筆貸款 (每月 10 號還款)**\n* 剩餘金額：`${l2_remain:,.0f}` 元\n* 月還款
額：`${18872:,.0f}` 元\n* 累計已還款：`{l2_pay_count}` 期\n*(基準起算點：2026-04-10)*")
```

```
    with loan_col3:
        st.success(f"**📊 總剩餘負債統計**\n* 總剩餘貸款：`${total_loan_balance:,.0f}` 元\n* 當前安全維持率分母：`${total_loan_balance:,.0f}`")

    st.markdown("---")

    st.subheader("🎯 核心資產再平衡與偏離度檢查")
    df_portfolio['目前投資占比'] = df_portfolio['當前市值'] / total_market_value if total_market_value > 0 else 0
    df_portfolio['偏離度 (Diff)'] = df_portfolio['目前投資占比'] - df_portfolio['核心權重']

    def generate_advice(row):
        if row['偏離度 (Diff)'] > 0.05:
            return f"⚠️ 建議減碼 {row['標的名稱']}"
        elif row['偏離度 (Diff)'] < -0.05:
            return f"🛒 建議加碼 {row['標的名稱']}"
        else:
            return "✅ 權重正常"

    df_portfolio['買賣建議'] = df_portfolio.apply(generate_advice, axis=1)

    st.dataframe(df_portfolio[['標的名稱', '核心權重', '單位現價', '持有數量', '當前市值', '目前投資占比', '偏離度 (Diff)', '買賣建議']].style.format({
        '核心權重': '{:.1%}',
        '單位現價': '${:,.2f}',
        '持有數量': '{:.0f}',
        '當前市值': '${:,.2f}',
        '目前投資占比': '{:.1%}',
        '偏離度 (Diff)': '{:+.1%}'
    }))

    st.markdown("---")

    # 歷史總資產趨勢追蹤
    st.subheader("📈 歷史總資產趨勢追蹤")
    if not df_history.empty:
        df_history['日期'] = pd.to_datetime(df_history['日期'])
        df_history = df_history.sort_values(by="日期")

        time_option = st.radio(
            "選擇顯示的時間區間：",
            ["近 7 天", "近 30 天", "近 180 天", "今年以來 (YTD)", "全部顯示", "自訂日期範圍"],
            horizontal=True,
            key="dashboard_time_range"
        )

        today = datetime.now()
        df_filtered = df_history.copy()
```

```
    if time_option == "近 7 天":
        start_date = today - timedelta(days=7)
        df_filtered = df_history[df_history['日期'] >= pd.to_datetime(start_date)]
    elif time_option == "近 30 天":
        start_date = today - timedelta(days=30)
        df_filtered = df_history[df_history['日期'] >= pd.to_datetime(start_date)]
    elif time_option == "近 180 天":
        start_date = today - timedelta(days=180)
        df_filtered = df_history[df_history['日期'] >= pd.to_datetime(start_date)]
    elif time_option == "今年以來 (YTD)":
        start_date = datetime(today.year, 1, 1)
        df_filtered = df_history[df_history['日期'] >= pd.to_datetime(start_date)]
    elif time_option == "自訂日期範圍":
        col_date1, col_date2 = st.columns(2)
        with col_date1:
            start_date_input = st.date_input("開始日期：", today - timedelta(days=30))
        with col_date2:
            end_date_input = st.date_input("結束日期：", today)
        df_filtered = df_history[(df_history['日期'] >= pd.to_datetime(start_date_input)) &
(df_history['日期'] <= pd.to_datetime(end_date_input))]

    if len(df_filtered) > 1:
        df_filtered = df_filtered.sort_values(by="日期").copy()
        last_idx = df_filtered.index[-1]
        df_filtered.loc[last_idx, '總資產金額'] = df_filtered.iloc[-2]['總資產金額']
        df_filtered.loc[last_idx, '每日報酬率'] = df_filtered.iloc[-2]['每日報酬率']

    if not df_filtered.empty:
        fig = make_subplots(specs=[[{"secondary_y": True}]])
        fig.add_trace(
            go.Bar(
                x=df_filtered["日期"],
                y=df_filtered["總資產金額"],
                name="總資產金額 (TWD)",
                marker=dict(color="#29B6F6", opacity=0.8)
            ),
            secondary_y=False
        )
        fig.add_trace(
            go.Scatter(
                x=df_filtered["日期"],
                y=df_filtered["每日報酬率"],
                name="累積報酬率 (%)",
                mode="lines+markers",
                line=dict(color="#FFB300", width=2, dash="dash"),
                marker=dict(size=5)
            ),
```

```
            secondary_y=True
        )
        fig.update_layout(
            title_text=f"資產總額與報酬率綜合成長曲線 ({time_option})",
            hovermode="x unified",
            legend=dict(orientation="h", yanchor="bottom", y=1.02, xanchor="right", x=1),
            margin=dict(l=50, r=50, t=80, b=50)
        )
        fig.update_yaxes(title_text="<b>總資產金額 (TWD)</b>", secondary_y=False)
        fig.update_yaxes(title_text="<b>累積報酬率 (%)</b>", tickformat=".1%",
secondary_y=True)
        st.plotly_chart(fig, use_container_width=True)

# ==========================================
# 功能二：✍️ 每日資產動態輸入
# ==========================================
elif menu == "✍️ 每日資產動態輸入":
    st.title("✍️ 每日資產金額輕鬆記")
    try:
        df_history = conn.read(worksheet="daily_asset_history", ttl=0)
        df_portfolio = conn.read(worksheet="portfolio_config", ttl=0)
    except Exception as e:
        st.error(f"❌ 讀取失敗: {e}")
        st.stop()

    df_history = df_history.dropna(subset=["日期"])
    df_portfolio = df_portfolio.dropna(subset=["標的名稱"])
    total_cost = df_portfolio['投資成本'].sum()

    with st.form("daily_input_form", clear_on_submit=True):
        input_date = st.date_input("選擇紀錄日期：", datetime.now())
        input_amount = st.number_input("今日結算總資產金額 (TWD)：", min_value=0.0,
step=1000.0, format="%.2f")
        submit_button = st.form_submit_button(label="🚀 提交並儲存至 Google Sheets")

        if submit_button:
            date_str = str(input_date)
            df_history['日期'] = df_history['日期'].astype(str)

            df_history_temp = df_history.copy()
            df_history_temp['日期'] = pd.to_datetime(df_history_temp['日期'])
            df_history_temp = df_history_temp.sort_values(by="日期")
            df_history_temp['日期'] = df_history_temp['日期'].dt.strftime("%Y-%m-%d")

            df_yesterday = df_history_temp[df_history_temp['日期'] < date_str]
            if not df_yesterday.empty:
                last_amount = float(df_yesterday['總資產金額'].iloc[-1])
                daily_diff = input_amount - last_amount
```

```
            else:
                daily_diff = 0.0

            daily_roi = round((input_amount - total_cost) / total_cost, 4) if total_cost > 0 else 0.0
            input_amount_rounded = int(round(input_amount))
            daily_diff_rounded = int(round(daily_diff))

            new_data = {
                "日期": date_str,
                "總資產金額": input_amount_rounded,
                "每日增額": daily_diff_rounded,
                "每日報酬率": float(daily_roi)
            }

            if date_str in df_history['日期'].values:
                df_history.loc[df_history['日期'] == date_str, ["總資產金額", "每日增額", "每日報酬
率"]] = [
                    input_amount_rounded,
                    daily_diff_rounded,
                    float(daily_roi)
                ]
            else:
                df_history = pd.concat([df_history, pd.DataFrame([new_data])],
ignore_index=True)

            with st.spinner('正在寫入...'):
                conn.update(worksheet="daily_asset_history", data=df_history)
            st.success("🎉 成功同步至雲端試算表！數值已自動四捨五入為整數。")

    st.markdown("---")
    st.subheader("📋 歷史資產紀錄查詢與管理")

    search_option = st.radio(
        "選擇歷史紀錄查詢區間：",
        ["近 7 天", "近 30 天", "近 180 天", "今年以來 (YTD)", "全部顯示", "自訂日期範圍"],
        horizontal=True,
        key="history_search_range"
    )

    df_display = df_history.copy()
    df_display['日期'] = pd.to_datetime(df_display['日期'])
    df_display = df_display.sort_values(by="日期", ascending=False)

    today = datetime.now()

    if search_option == "近 7 天":
        start_date = today - timedelta(days=7)
        df_display = df_display[df_display['日期'] >= pd.to_datetime(start_date)]
```

```
    elif search_option == "近 30 天":
        start_date = today - timedelta(days=30)
        df_display = df_display[df_display['日期'] >= pd.to_datetime(start_date)]
    elif search_option == "近 180 天":
        start_date = today - timedelta(days=180)
        df_display = df_display[df_display['日期'] >= pd.to_datetime(start_date)]
    elif search_option == "今年以來 (YTD)":
        start_date = datetime(today.year, 1, 1)
        df_display = df_display[df_display['日期'] >= pd.to_datetime(start_date)]
    elif search_option == "自訂日期範圍":
        col_date1, col_date2 = st.columns(2)
        with col_date1:
            start_date_input = st.date_input("查詢開始日期:", today - timedelta(days=30),
key="query_start")
        with col_date2:
            end_date_input = st.date_input("查詢結束日期:", today, key="query_end")
        df_display = df_display[(df_display['日期'] >= pd.to_datetime(start_date_input)) &
(df_display['日期'] <= pd.to_datetime(end_date_input))]

    df_display['日期'] = df_display['日期'].dt.strftime("%Y-%m-%d")
    df_display['總資產金額'] = pd.to_numeric(df_display['總資產金額'],
errors='coerce').fillna(0).round().astype(int)
    df_display['每日增額'] = pd.to_numeric(df_display['每日增額'],
errors='coerce').fillna(0).round().astype(int)

    rows_per_page = 10
    total_rows = len(df_display)

    if total_rows > 0:
        total_pages = int(np.ceil(total_rows / rows_per_page))
        col_page_sel, col_page_info = st.columns([1, 4])
        with col_page_sel:
            current_page = st.number_input(f"頁碼 (共 {total_pages} 頁)", min_value=1,
max_value=total_pages, value=1, step=1)

        start_idx = (current_page - 1) * rows_per_page
        end_idx = start_idx + rows_per_page
        st.dataframe(df_display.iloc[start_idx:end_idx], use_container_width=True)
    else:
        st.info("ℹ️ 該時間區間內查無歷史紀錄。")

# ==========================================
# 功能三:⚙️ 投資標的持股管理
# ==========================================
elif menu == "⚙️ 投資標的持股管理":
    st.title("⚙️ 投資標的與持股數量管理")
    try:
        df_portfolio = conn.read(worksheet="portfolio_config", ttl=0)
```

```
except Exception as e:
    st.error(f"❌ 讀取失敗: {e}")
    st.stop()

df_portfolio = df_portfolio.dropna(subset=["標的名稱"])
st.subheader("✏️ 線上編輯持股資訊")
edited_df = st.data_editor(df_portfolio, num_rows="dynamic")

if st.button("💾 儲存並同步至 Google Sheets"):
    with st.spinner('正在儲存...'):
        conn.update(worksheet="portfolio_config", data=edited_df)
    st.success("💾 修改已同步！")
```